NetraTrust build pipeline

Oct 27, 2026 · @Mudit Kumar

Six research streams verified the stack against live package registries, and two of them executed the critical code paths rather than reading about them. **Four of your deck's assumptions did not survive**, and one of them — the hosting choice — would have deleted your ledger every fifteen minutes. The build window is Wednesday 09:00 to Friday 12:00, roughly 30 working hours across three people.

What changed from the deck

Deck said	Verification found	Do instead
TLS / mTLS for secure backend communication	uvicorn does **not expose the client certificate to the application** — `request.scope` was enumerated and contains no peer-cert key. You cannot check a cert against a site ID or bind it to a ledger entry. And mTLS cannot authenticate an event captured *while offline*, because no connection existed	**Skip mTLS.** Per-device Ed25519 payload signatures are strictly better here, not merely cheaper. Ship a `gen_certs.sh` and one slide line
EyeQ for image quality	EyeQ is **labels only** — 12,543 train / 16,249 test rows, measured directly from the CSVs. Images come from EyePACS, which is **89 GiB**. MCF-Net weights are dead: "no longer usable due to several version updates"	Use **QuickQual** (27 MB, verified live download, beats MCF-Net on every EyeQ metric with **zero training**). Work in a **Kaggle notebook** where EyePACS is already mounted

Deck said	Verification found	Do instead
OpenCV focus and exposure checks as the gate	**No published absolute Laplacian-variance threshold for fundus images exists.** The 2025 systematic review does not mention Laplacian variance and gives no numeric thresholds for any classical metric. The familiar `var < 100` is from a natural-image blog demo	Keep classical metrics as a **parallel reason-code channel** — *why* it failed and *what to retake* — not as the gate itself
Deploy somewhere free	**Fly.io removed its free tier. HF Docker Spaces now need a paid plan. Render free has no persistent disk** — the docs say filesystem changes are lost on restart or spin-down, and it spins down after 15 minutes idle. **Your ledger would be deleted roughly every 15 minutes**	**Fly.io at ~$2.34/month** (256 MB machine + 1 GB volume), auto-stop **off**, one machine. The only option with a durable local filesystem, which WAL mode requires. Railway Free is the zero-budget fallback

Two smaller corrections

Ben Graham preprocessing and CLAHE are not supported by evidence. A controlled ResNet-50 ablation found Ben Graham at 0.8260 and CLAHE at 0.8238, both *below* plain background-removal + resize at 0.8310. What actually matters is resolution: 128→512 buys **+0.164 kappa**, while 512→1024 buys **+0.013** for four times the compute. **512px is the efficient frontier.**

A risk–coverage curve takes a horizontal reference line, at the no-abstention risk — not the `y=x` diagonal. Only calibration curves take the diagonal. Putting a diagonal on a risk–coverage plot is a methodological tell a technical judge will catch.

One thing the deck got right that is worth defending

Streamlit. It was re-examined against Vite + React + shadcn and **wins on arithmetic**: the React path costs roughly 35 hours against a ~24 hour budget. The two classic objections are now obsolete — `st.navigation(position="top")` removes the sidebar tell, and

`config.toml` themes Plotly series colours centrally, removing the default-colourway tell. The reasoning worth repeating to a judge: *a plain app whose hash chain genuinely breaks on stage is more credible than a polished one with two views stubbed and a faked tamper demo.*

Tonight

Ordered by what blocks Wednesday morning. Items 1–4 are the ones that stop the build dead if they are not done.

- [ ] **1. Open a Kaggle notebook with APTOS 2019 and EyePACS attached.** This is the single biggest schedule risk removed. EyePACS is 89 GiB; you are not downloading it to a hostel connection. Both datasets are already mounted inside Kaggle. Verify the notebook starts and both paths resolve, then stop. *~20 min.*
- [ ] **2. Click the RETFound model agreement on Hugging Face.** The weights are gated under `YukunZhou/*` (CC BY-NC 4.0) and **it is not documented whether approval is automatic or manual**. If it is manual and you click on Wednesday, it will not arrive in time. Click tonight as insurance — you may not need it. *~5 min.*
- [ ] **3. Test that the QuickQual pickle loads.** It is a 2023-era scikit-learn pickle and you are on 1.9.1. If it fails, fall back to **QuickQual-MEME**, whose ten parameters are printed in the paper so nothing needs downloading. Knowing which of these you are using decides Wednesday's first task. *~15 min.*
- [ ] **4. Pick Python 3.12 or 3.13 and build the venv from `requirements.txt`.** numpy 2.5.4 and scipy 1.18.1 now require ⩾3.12; Streamlit requires ⩾3.11. **A team on 3.11 silently gets old numpy** and will spend Thursday debugging it. *~20 min.*
- [ ] **5. Freeze the splits and commit them.** Run `scripts/make_splits.py` against your manifest. Patient-level, source-preserved, seeded. This is the highest-leverage hour in the whole build, because patient duplication across splits is invisible in the results — it just makes everything look better. *~45 min.*
- [ ] **6. Verify two citations in the deck, or remove them.** The **42% cataract figure** could not be verified (the source served a CAPTCHA), and the **AIOS/VRSI 2021 consensus statement** could not be opened. Neither is disproven — but your problem statement rests on the first. Mihir opens both links and confirms title, authors and figure. *~30 min.*
- [ ] **7. Decide the backbone now: `convnext_tiny.in12k_ft_in1k`.** This is not a Wednesday decision because **a ViT doubles the Grad-CAM wiring time** on Friday. Settle it tonight. *~5 min.*

- [ ] **8. Create the repo, three branches, and the ownership lanes.** Drop in `CLAUDE.md`, `SPEC.md`, `requirements.txt`, `.claude/settings.json`, `.claude/rules/ml.md`, `netra/eval/metrics.py`, `scripts/make_splits.py`. Push. *~30 min.*
- [ ] **9. Read `SPEC.md` aloud together once.** Twenty minutes. The API contract is the thing that lets three people build in parallel without talking; if one of you disagrees with it, tonight is free and Thursday is not. *~20 min.*

Total: about 3 hours 30 minutes. If you only have time for four, do 1, 2, 4 and 5.

One thing deliberately not on this list

Do not start writing application code tonight. The documented failure mode of a three-day AI-assisted build is drift from a spec that was never written down, and you are currently ahead of schedule on exactly the artefact that prevents it.

The verified stack

All versions read from PyPI and npm on 27 October 2026. Everything recommended is OSI-licensed.

Models and data

Component	Choice	Why	Access
Quality gate	**QuickQual** — frozen `densenet121.tv_in1k` @512px + SVM	EyeQ accuracy **0.886**, AUC 0.9687, kappa 0.811 — beats MCF-Net on every metric, **zero training**. 27 MB, download verified live	Direct
Quality gate fallback	**QuickQual-MEME**	10 parameters printed in the paper — nothing to download at all	None needed
Quality gate alt	**EFIQA** (MIT)	87.55% but produces a **region overlay** showing where the image is bad — serves recapture routing	HF + GitHub releases

Component	Choice	Why	Access
Quality labels	**EyeQ CSVs**	12,543 train / 16,249 test, measured directly. The README's own table disagrees with its CSVs — trust the CSVs	GitHub (labels only)
DR backbone	**`convnext_tiny.in12k_ft_in1k`** @512px	Only option with two independent lines of DR-specific support	timm
DR fallback	RETFound **APTOS checkpoint** from `BENCHMARK.md`	Already fine-tuned, Google Drive, **no gate**. Publishes no metrics — you must re-measure	Direct
DR data	**APTOS** via gate-free HF mirror `bumbledeep/aptos`	3.66k rows, labelled 0–4, no Kaggle rules click-through	Direct
Degradation	`util/cataract_simulation.py` from `liamheng/arcnet-medical-image-enhancement`	Pure cv2/numpy, CPU-only, 16 variants per image. **Severity is parameterised**, so it calibrates your thresholds	GitHub

Three gotchas in the degradation script, found by reading its source: the README's path is wrong (`util/` , not `utils/` , and the filename is not misspelled as the README has it); `IMAGE_DIR` and `OUTPUT_DIR` are assigned twice with the author's hard-coded Windows paths winning; and output is pix2pix-style side-by-side pairs that must be split.

Python packages

Purpose	Package	Version	Licence
Modelling	`torch` / `torchvision` / `timm`	2.14.1 / 0.29.1 / 1.0.30	BSD / Apache-2.0

Purpose	Package	Version	Licence
Conformal	`mapie`	**1.6.0**	BSD-3 — deps are only numpy/sklearn/scipy
Classical ML	`scikit-learn` / `numpy` / `scipy`	1.9.1 / 2.5.4 / 1.18.1	BSD
API	`fastapi` / `starlette` / `pydantic` / `uvicorn`	0.162.0 / **1.6.0** / 2.14.1 / 0.54.0	MIT / BSD
ORM + ledger	`sqlmodel` + raw `sqlite3`	0.0.48	MIT
Canonical JSON	`rfc8785`	0.1.4	MIT
Crypto	`cryptography`	50.0.2	Apache-2.0 / BSD
UI	`streamlit` / `plotly`	1.66.1 / 7.1.0	Apache-2.0 / MIT
Explainability	`grad-cam` (import `pytorch_grad_cam`)	1.5.7	MIT
Interop	`fhir.resources`	8.3.0	BSD

Pin `starlette==1.6.0`. It crossed into 1.x on 24 October, one day after the current FastAPI release, and FastAPI's upper bound is unpinned. 1.6.0 is verified working; 1.10.0 is not.

Banned

Package	Reason
`alibi-detect`	**Business Source License 1.1** — PyPI itself classifies it as proprietary
`sdv`	**BUSL-1.1** — takes the whole SDV synthetic-data ecosystem off the table
`puncc`	Pins `numpy<2.0.0`
`pymerkle`	**GPL-3.0-or-later** (viral)

Package	Reason
`aiosqlite`	Unmaintained since Dec 2025, and SQLite has one writer regardless
Any LLM or RAG component	See below

Why there is no LLM

GPT-4o was tested on **exactly this task** — 2,500 EyePACS fundus photographs, 500 per grade — and reached **47.7% sensitivity / 73.8% specificity** at the referral threshold. Approved CNN devices reach IDx-DR ~87%/90% and EyeArt 94.4%/91.1%. *Clinical Ophthalmology* 2025;19:3103–3112.

Every candidate use was tested on its merits and failed. Referral letters → templates, because you cannot QA languages you do not read and mistranslated urgency is an unsupervised safety event. Site summaries → deterministic templating, because a summary is read *instead of* the table, so a misstated number is believed. Guideline lookup was the strongest candidate and still fails: the actionable content is a 10–40 row referral-interval table that belongs in reviewed YAML.

Make this a **positive claim to judges, not an apology**: *the generative model was tested on this task and reached 47.7% sensitivity where approved devices reach 87–94%, so it appears only where it cannot touch a decision — and there is no such place.*

Repo, ownership, and branches

Layout

```
netratrust/
  CLAUDE.md                 conventions + library gotchas (<200 lines)
  SPEC.md                   the contract - read before writing code
  requirements.txt          pinned, do not bump during the build
  .python-version           3.12
  .claude/
    settings.json           Stop hook runs pytest; deny rules on data/ and
keys/
    rules/ml.md             path-scoped rules for models/ and eval/
  config/
    thresholds.yaml         every threshold lives here, never inline
  data/
```

```
    splits/                  FROZEN. train.json calib.json test.json
meta.json
  scripts/
    make_splits.py           run once, commit output, never rerun
    gen_certs.sh             the mTLS story we tell but do not ship
  netra/
    models/      MIHIR       quality gate, DR classifier, conformal wrapper
    eval/        MIHIR       metrics.py - hand-written, single source of
numbers
    api/         MUDIT       FastAPI routes
    ledger/      MUDIT       hash chain, Ed25519, verify, checkpoint
    db/          MUDIT       SQLModel tables, referral state machine
    ui/          BHAVY       Streamlit pages
  notebooks/     MIHIR       Kaggle work, exported artefacts only
  tests/         everyone    mirrors the package layout
```

Ownership

The lanes are **disjoint directories**, which is what makes three people and three agents possible in one repo. Nobody edits outside their lane; `SPEC.md` and `requirements.txt` change only by agreement, in their own commit.

Person	Owns	Wednesday deliverable
Mudit (lead, backend)	`api/`, `ledger/`, `db/`	Ledger appends, verifies, and survives a tamper attempt
Bhavy (frontend)	`ui/`	Capture page posts to a real endpoint and renders a verdict
Mihir (research/ML)	`models/`, `eval/`, `notebooks/`	Quality gate returns a verdict on a real image

Branches

Deliberately boring. Three branches named for people, not features. **Merge to `main` twice a day — after lunch and before stopping.** No git-flow, no rebase, no long-lived feature branches.

The reason for twice-daily rather than continuous: a three-day build has no time to recover from a merge that went wrong overnight, and no time to resolve a week of divergence on Friday morning.

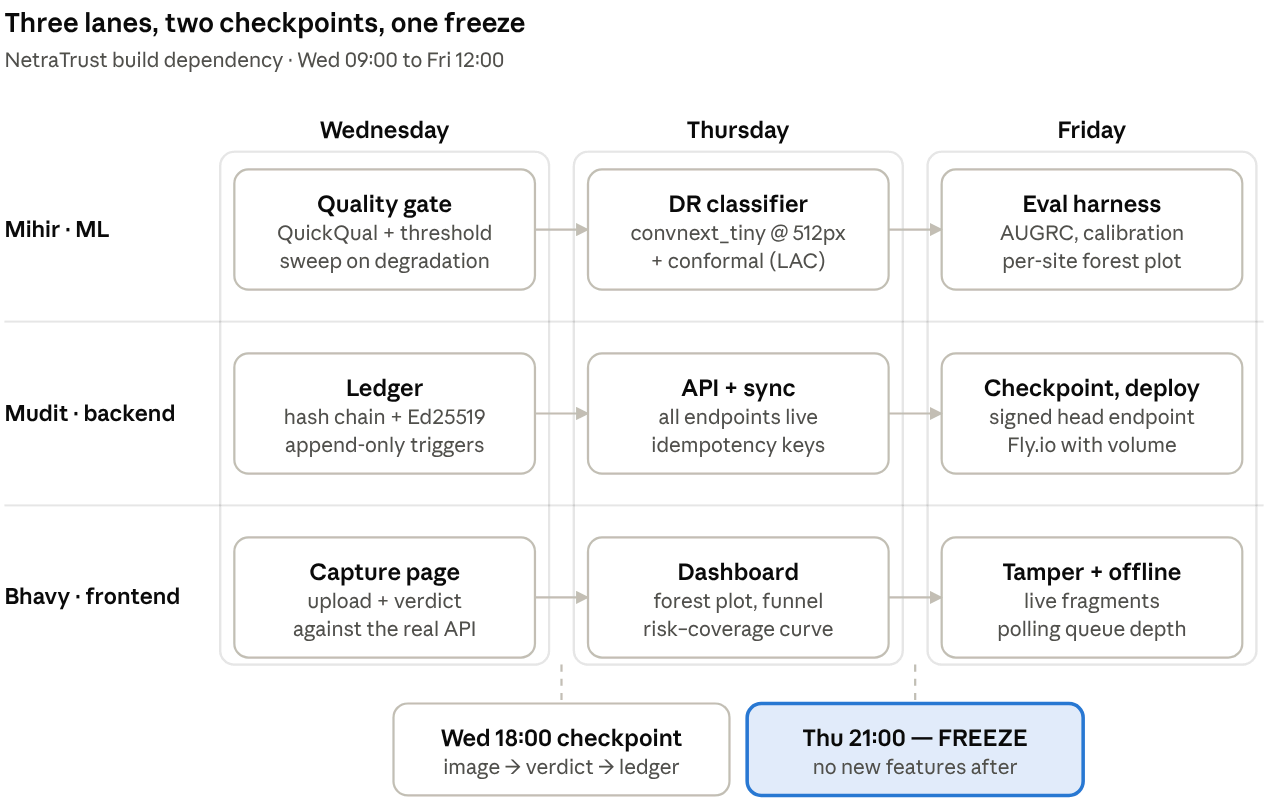
How to use Claude Code across three days

- **Spec-first, then a fresh session.** The documented flow is to settle the spec, then **start a new session to build from it** — the context that wrote the spec is not the context that should implement it.
- **CLAUDE.md is for conventions, not architecture.** The docs explicitly retire "describe your file layout in CLAUDE.md". Architecture lives in `SPEC.md`, which you point at.
- **Hooks are deterministic; CLAUDE.md is not.** The `Stop` hook running `pytest -q` is enforcement. A line in CLAUDE.md is a guideline the model is allowed to decide does not apply. Put anything that must hold in a hook or a deny rule.
- **After two failed corrections, `/clear` and write a better prompt** rather than attempting a third. This is in the official guidance and it is the single most useful habit for a time-boxed build.
- Each person runs their own session in their own lane. Do not run parallel agents on the same directory.

The failure no hook catches

Evaluation drift: generated code that passes its unit tests while computing metrics on the wrong split, or tuning the abstention threshold on test. No automated guard exists for this. The mitigation is structural and it is already done — `netra/eval/metrics.py` is hand-written, the splits are frozen and committed, and `.claude/rules/ml.md` is path-scoped so it loads whenever anything under `models/` or `eval/` is touched.

The dependency graph



build dependency · 3 lanes × 3 days, 2 gates

The three lanes only touch at the two gates, which is what lets three people and three agents work in one repo without colliding. Everything Bhavy builds on Wednesday runs against a real endpoint, so there is no integration day — integration happens continuously, and the Wednesday checkpoint proves it.

Wednesday

Goal: one image travels end to end by 18:00. Not a good result — a complete path. Quality gate → verdict → ledger entry → visible in the UI.

Time	Mihir · ML	Mudit · backend	Bhavy · frontend
09:00–10:30	Load QuickQual, run it on 20 real APTOS images, confirm sane verdicts	SQLite schema + append-only triggers. Write the ledger append and verify functions	Streamlit skeleton, `st.navigation(position="top")`, five empty pages, theme in `config.toml`

Time	Mihir · ML	Mudit · backend	Bhavy · frontend
10:30–12:30	Run the cataract simulator over a sample. Split the pix2pix side-by-side output pairs	Ed25519 keygen, sign the 32-byte entry hash, `rfc8785` canonicalisation on both write and verify	Capture page: file upload, POST to `/v1/encounters`, render the returned verdict
12:30–13:30	*Lunch.* ***Merge to `main`.***		
13:30–15:30	**The threshold sweep.** Degrade images at increasing severity, measure where DR accuracy collapses. That curve sets usable/borderline/reject	`POST /v1/encounters` live, appending to the ledger. Return `{encounter_id, ledger_idx}`	Verdict rendering with cause category; badge styling; the image panel
15:30–17:00	Wire the gate behind a function the API can import. Write `config/thresholds.yaml`	`GET /v1/ledger/verify`. Then the **tamper script**: trigger blocks → drop trigger → mutate → three layers fail	Abstention queue page reading from the API
17:00–18:00	— **Everyone: integration.** Image in the UI → gate verdict → ledger row → verify passes → tamper fails —		
18:00	**Checkpoint. Merge to `main`.** If the path is not complete, cut scope tonight, not Friday		

The sweep is the day's real work

Everything else on Wednesday is plumbing. The threshold sweep is what converts "we picked 0.7" into "we measured where the classifier breaks and set the threshold there" — and because the simulator's severity is parameterised, you can actually do this. It is the most defensible design decision available to you in three days, and it is the answer to the first question a technical judge will ask about the gate.

If you are behind at 18:00

Cut in this order: the cause-category UI, the abstention queue page, the classical reason-code channel. **Do not cut** the ledger verify or the threshold sweep — they are the two things that are hard to add later and are the spine of the demo.

Thursday

Goal: the headline experiment exists, and the system is feature-complete by 21:00.

Time	Mihir · ML	Mudit · backend	Bhavy · frontend
09:00–12:00	Fine-tune `convnext_tiny.in12k_ft_in1k` @512px on the frozen train split. Background removal + resize only — no Ben Graham, no CLAHE	Referral state machine + `POST` / `PATCH` `/v1/referrals`, every transition appended to the ledger	Dashboard page: per-site **forest plot** of Wilson intervals against the pooled line
12:00–13:00	*Lunch.* ***Merge to*** `main`***.***		
13:00–15:00	**The headline experiment.** Ungated specificity on all images vs gated specificity on admitted images, abstention rate reported beside it. Plus the classical baseline on the same split	Offline queue: `Idempotency-Key`, 409 in-flight, 422 key reuse with different payload. `GET` `/v1/queue/depth`	Risk–coverage curve (**horizontal** reference line), calibration curve (`y=x`), referral funnel via `go.Funnel`
15:00–17:30	MAPIE conformal wrapper, `conformity_score="lac"`, `prefit=True`. Abstain when the set spans more than one clinical action	Connectivity flag held **server-side** so the demo survives a page navigation and a second tab	Ledger page: verify status, and the tamper demo wired to the backend script
17:30–19:30	Calibration: Brier, NLL, slope, intercept, temperature scaling. ECE secondary with its binning stated	`GET` `/v1/metrics/sites` returning distribution rows, never a mean	Offline indicator: `st.fragment(run_every="1s")` polling queue depth
19:30–21:00	— **Everyone: integration, then full dry run of the demo** —		
21:00	**FREEZE. Merge to** `main`**.** No new features after this point		

Why the freeze is Thursday night and not Friday

The common failure is a team still adding features on Friday morning, discovering an integration break at 11:00, and presenting something that half-works. Freezing Thursday night gives you Friday morning to find what is broken while there is still time to fix it, and Friday afternoon to rehearse.

A freeze means: no new endpoints, no new pages, no new model runs. Bug fixes, polish, and the optional Friday extras are allowed because they cannot break what already works — and if one does, you revert it rather than debug it.

Two Thursday traps

The offline demo must show per-item identity, not an animation. Row IDs leaving a table, `queued_at` → `synced_at` pairs, an `attempts: 2` retry column. That is what separates a real queue from a spinner, and judges look for exactly this.

The abstention threshold is set on the calibration split. If anyone tunes it against test numbers — even once, even casually, even "just to see" — the headline result is no longer reportable. This is the single rule most likely to be broken accidentally under time pressure.

Friday

Hard stop at 12:00. Everything after noon is rehearsal.

Time	Who	What
09:00–10:00	Mudit	Deploy to Fly.io: 256 MB machine, 1 GB volume, **auto-stop off**, one machine only. Verify the ledger survives a restart
09:00–10:00	Mihir	Final eval run on the test split. **This is the only time test is touched.** Generate every figure
09:00–10:00	Bhavy	Load the final numbers into the dashboard; check every chart renders with real data
10:00–11:00	Mudit	`GET /v1/ledger/checkpoint` — signed head. **30 minutes, best value-per-minute in the build**

Time	Who	What
10:00–11:00	Mihir	Grad-CAM if the backbone is a CNN (~2 h; 3–4 h for a ViT, which is why this was decided Tuesday)
10:00–11:00	Bhavy	The hero panel: 2–3 hours of hand-authored HTML via `st.html` so the app does not read as templated
11:00–12:00	All	Optional: FHIR `Observation` with the ledger `entry_hash` embedded as an extension (45 min)
12:00	—	**HARD STOP. Nothing is edited after this.**
12:00–14:00	All	Full dry run, twice. Time it. Fix only what breaks
14:00–15:00	All	Write the limitations slide. Rehearse the judge Q&A

The FHIR move is cleverer than it looks

Embedding the ledger `entry_hash` as a FHIR `extension` makes your interoperability story and your tamper-evidence story the **same story**, for 45 minutes. One warning: **do not invent a LOINC code.** An invented clinical code is exactly what a medical judge catches. Use a plainly-labelled local code system unless Mihir verifies a real one.

The three-minute demo

Time	Show	Say
0:00–0:20	**96.01% → 64.7%**	"One algorithm, one 2025 study. Validation in two Punjab cities, field at a health centre in Moga. The algorithm didn't change. The images did."
0:20–0:45	Clean image: gate admits, grade returned, referral created	"Normal path, end to end, written to the ledger."
0:45–1:25	Cataract-degraded image **twice** — gate off: a *confident* grade. Gate on: abstains, cause `media opacity`, routes to ophthalmology	"With the gate off the model isn't uncertain. It's confident and wrong. With it on, it abstains and refers for a cataract assessment — itself a treatable finding."

Time	Show	Say
1:25–1:55	Ungated vs gated specificity, **abstention rate beside it**, risk–coverage curve, calibration curve, and the classical baseline on the same split	"Here's the cost: this percentage leave without an answer. We report it next to the gain. And here's the rule-based baseline — what the learned model has to beat to justify itself."
1:55–2:15	Forest plot, three sites, worst one named	"We never show a mean. Pooled validation gives an average AUROC drop of 0.037 — but it exceeds 0.05 in half of studies and reaches 0.391. The average hides the site that fails."
2:15–2:35	Pull the network — queue fills with row IDs, reconnect drains it. Then the **three-beat tamper**: trigger blocks → drop trigger → payload hash, entry hash, signature each fail	"The deployment environment loses power and connectivity. And the FHIR audit standard protects integrity by *advising* servers not to accept updates. That's a policy. This is a control."
2:35–2:50	Funnel, 64 → 28 → 9	"Of 64 referred, 28 reachable, 9 attended. Our gate improves the input; this registry addresses the bigger failure. This funnel is a simulation parameterised by that study, not something we measured."
2:50–3:00	Limitations slide, **read aloud**	"We don't claim better vision outcomes. We don't claim to beat any commercial algorithm. Our comparison is against our own ungated system."

The one rehearsal note that matters most: the abstention rate must be on screen at the same moment as the specificity gain. A team that shows only the gain has demonstrated it does not understand its own trade-off, and that is the first thing a competent judge will probe.

On the tamper demo, sequence matters. Your own append-only triggers will block the demo, so show that *first* — `sqlite3.IntegrityError: ledger is append-only` — then drop the trigger. Three independent layers failing in turn is what separates this from a checksum column.

Acceptance gates

A component is done when all three hold: it has a test that fails if the behaviour regresses, its numbers come from `netra/eval/metrics.py` rather than inline computation, and it appears in the demo script — or it is cut.

Component	Done means
Quality gate	Returns `{usable|borderline|reject}` + cause + score on a real image. Thresholds come from `config/thresholds.yaml`, and that file records the sweep that produced them
Degradation sweep	A curve exists showing DR accuracy against degradation severity, and the chosen thresholds are marked on it
DR classifier	QWK on the frozen test split, reported once. **Expect 0.88–0.92.** Above 0.95 means a leaked split — stop and find the leak
Conformal wrapper	Prediction sets at a stated confidence level, with abstention triggered by clinical-action span, not set size
Headline experiment	A single table: ungated specificity on all images, gated specificity on admitted images, abstention rate, classical baseline. All four numbers, one table
Calibration	Brier and NLL as primary; slope and intercept; ECE only with its binning scheme printed beside it
Selective prediction	Risk–coverage curve with a **horizontal** reference, plus AUGRC (not AURC)
Ledger	`verify` returns `{ok: true}` on a clean chain and `{ok: false, first_bad_index: n}` after a mutation, with the three layers failing in order
Checkpoint	`GET /v1/ledger/checkpoint` returns a signed head that an external party could compare against an earlier one

Component	Done means
Offline sync	Network off → queue depth rises with visible row IDs → network on → drains. `attempts` increments on retry. Survives a page navigation and a second tab
Site registry	A site that has not passed its audit does not have its grades counted, and the API enforces this rather than the UI
Dashboard	Per-site forest plot of Wilson intervals, never a mean. Every proportion with n < 30 shows its count and interval
Deployment	The app is reachable at a URL, and the ledger still verifies after a machine restart

Three numbers that must never appear

1. **A local split score beside a public leaderboard score.** The APTOS leaderboard top figure is unverified in this research anyway — check it yourself before quoting any winning number, or quote none.
2. **A bare percentage for n < 30.** Count plus Wilson interval, always.
3. **A mean across sites as the headline.** The entire argument for per-site reporting is that the mean hides the failure.

What will go wrong

Problem	Likelihood	What to do
The QuickQual pickle will not load under scikit-learn 1.9.1	Moderate — it is a 2023-era pickle	Switch to QuickQual-MEME. Ten parameters, printed in the paper, nothing to download. You tested this Tuesday so you already know
RETFound's gate is manual and does not approve in time	Unknown — undocumented	You are not depending on it. `convnext_tiny` is the plan; RETFound was insurance

Problem	Likelihood	What to do
`brier_score_loss(y, y_prob=p)` raises TypeError	**High** — assistants emit the old keyword confidently	It is `y_proba=`. This is in CLAUDE.md. When it happens, you will recognise it in seconds instead of twenty minutes
The ledger chain breaks for no apparent reason	Moderate	A float. `rfc8785.dumps({"n": 1.0})` gives `{"n":1}` while `json.dumps` gives `{"n":1.0}`. Check that both sides canonicalise identically and that no float reached a payload
An integer raises in `rfc8785`	Moderate	Above 2^{53}. Timestamps and IDs are strings
FastAPI is mysteriously slow or deadlocks	Moderate	`async def` route handlers with blocking SQLite. Make them sync `def`
The deployed ledger is empty after lunch	High *if you ignore the hosting advice*	Render free has no persistent disk and spins down after 15 minutes idle. Fly.io with a volume, auto-stop off
Merge conflict on Friday morning	Moderate	Lanes are disjoint directories and you merge twice daily. If it happens anyway: the person whose lane it is resolves it, nobody else touches it
Two agents edit the same file	Moderate	One session per person, each in their own lane. Do not run parallel agents on one directory

Problem	Likelihood	What to do
The model is still training at 21:00 Thursday	**High** — this is the classic	The freeze is why it is Thursday and not Friday. Ship the checkpoint you have. A 0.88 QWK model with an honest evaluation beats a 0.92 model you cannot demo
The demo breaks during the presentation	Moderate	Rehearse twice on Friday afternoon against the deployed URL, not localhost. Have a recorded fallback of the tamper sequence

The failure that would actually cost you the hackathon

Not a bug. **Evaluation drift** — reporting a number computed on the wrong split, or an abstention threshold tuned on test. It passes every test, looks like a strong result, and collapses under one informed question. It is also the thing AI-assisted coding is most likely to introduce, because the wrong version looks exactly like the right version.

The structural defences are already in place: frozen committed splits, a hand-written metrics module, and path-scoped rules that load whenever `models/` or `eval/` is touched. **The human defence is that Mihir reads every number before it goes on a slide and can say which split produced it.**

Known unknowns to verify yourselves

This research was thorough but not complete, and these three gaps are yours to close:

- **The 42% cataract figure and the AIOS/VRSI 2021 statement** could not be verified — the sources served CAPTCHAs. Neither is disproven. Open them yourselves.
- **GitHub, Hugging Face and Kaggle were partly unreachable** during this research, so every "no existing implementation found" is a weak negative. One afternoon of repository searching either strengthens your novelty claim or tells you to build on something.
- **No citable wall-clock training times on a T4 or P100 exist** in any retrieved paper — every one omits hardware and timing. The Wednesday and Thursday estimates are arithmetic from dataset size and published FLOPs, not measurements. Treat them as

planning estimates and check your first epoch against them.